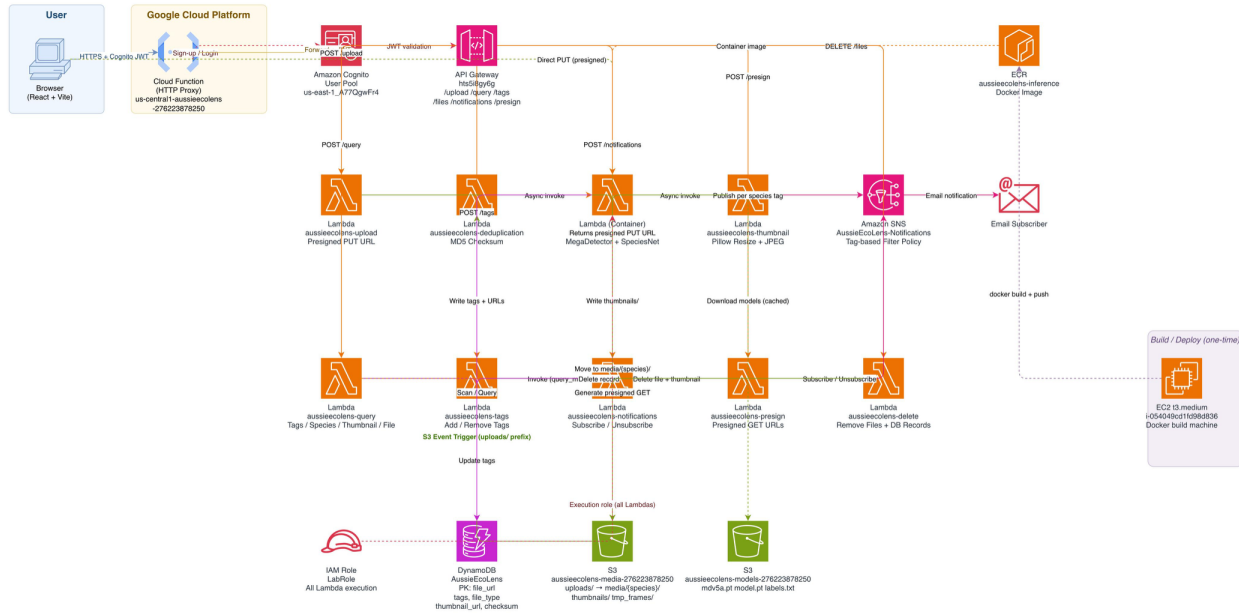


FIT5225 Cloud Computing and Security — Assignment 2

AussieEcoLens — Team Report

1. Multi-Cloud Architecture



AussieEcoLens is a serverless wildlife observation platform built across Amazon Web Services (AWS) and Google Cloud Platform (GCP). The architecture diagram is included above.

AWS serves as the primary cloud provider, hosting all core compute, storage, database, and messaging services. GCP hosts a single HTTP Cloud Function (us-central1-aussieecolens-276223878250) that acts as a secure proxy, forwarding authenticated requests from the React and Vite frontend to AWS API Gateway with the Cognito JWT token passed unchanged in the Authorization header. This satisfies the multi-cloud requirement while keeping the security perimeter on AWS.

Authentication is handled entirely through Amazon Cognito. Users register with email verification, receive a JWT on login, and all subsequent API calls are validated by a Cognito authoriser attached to every API Gateway route. The frontend communicates exclusively through the GCP proxy.

When a file is uploaded, the frontend calls POST /upload on API Gateway to receive a presigned S3 PUT URL, then uploads the file directly to the uploads/ prefix of the media bucket using that URL. An S3 event trigger scoped to the uploads/ prefix invokes the deduplication Lambda, which computes an MD5 checksum and rejects duplicates by deleting the file and returning a 409. For

new files, the deduplication function writes an initial DynamoDB record and asynchronously invokes the inference Lambda.

The inference Lambda runs as a Docker container image pulled from Amazon ECR. It executes MegaDetector (mdv5a.pt) to detect animal bounding boxes, then routes detections through the fine-tuned SpeciesNet classifier (model.pt) to produce species tags. All model weights are stored in a dedicated S3 models bucket and downloaded to /tmp/models/ on first invocation, where they are cached for the lifetime of the Lambda container. Class labels are loaded dynamically from labels.txt in the same bucket at runtime, meaning the model is fully swappable without any code changes. Detected tags, the file URL, file type, and checksum are written to DynamoDB. The file is then moved from uploads/ to a species-specific path under media/ in the media bucket.

The thumbnail Lambda runs asynchronously after inference. For images, it generates a resized JPEG using Pillow with aspect ratio preserved. For videos, it extracts one frame per second using OpenCV, selects the frame with the highest detection confidence as the thumbnail source, uploads it to thumbnails/ in S3, and cleans up the temporary frames. Amazon SNS publishes a notification per detected species using filter policies, so subscribers receive email alerts only for their chosen species.

The media bucket is fully private with S3 Block Public Access enabled. All media access from the frontend goes through a dedicated presign Lambda (aussieecolens-presign), which accepts a list of raw S3 URLs via POST /presign on API Gateway and returns time-limited GET presigned URLs valid for one hour. This endpoint is protected by the same Cognito authoriser as all other routes. DynamoDB records store the original raw S3 paths, and the presign endpoint resolves them on demand, so no data migration was required when the access model changed.

2. Team Contribution

Name	Student ID	Contribution	Description
Bharath Arun Gandhimani	35501308	25%	Designed and implemented the complete AWS backend: all nine Lambda functions (deduplication, inference, thumbnail, upload, query, tags, notifications, presign, delete), S3 bucket configuration including Block Public Access and presigned URL architecture, EC2-based Docker build pipeline. Conducted end-to-end integration testing across the full inference pipeline.
Gokhulraj Saravanan	35464909	25%	Designed and built the complete React and Vite frontend, including all seven UI panels. Integrated AWS Amplify for Cognito authentication and implemented presigned URL handling across all panels that display or link S3 media, including query results, thumbnail

			lookups, and reverse search. Connected all panels to the backend API service layer.
Premkumar Satish	35373598	25%	Implemented the GCP Cloud Function HTTP proxy that forwards authenticated frontend requests to AWS API Gateway with Cognito JWT tokens intact. Enabled API Gateway route configuration. Assisted in designing and finetuning the UI. Performed UI testing.
Melissa Maria Augustine	35707178	25%	Configured DynamoDB schema and table, SNS topic and species-based filter policies, ECR container image repository, and Cognito user pool setup. Conducted end-to-end testing of the notifications pipeline including subscription, filter policy validation, and email delivery confirmation. Contributed to team report preparation.

3. User Guide

Prerequisites: A modern web browser and a valid email address for account registration.

Step 1 — Local Setup. Ensure Node.js (v18 or above) is installed. Clone the repository from <https://github.com/bgan0012/AussieEcoLens> and navigate to the frontend directory. Run:

```
npm install
```

```
npm run dev
```

The application will be available at <http://localhost:5173>

Step 2 — Register an account. Click Sign Up and enter your email address, first name, last name, and a password. Cognito will send a verification email. Enter the confirmation code when prompted.

Step 3 — Sign in. Use your registered email and password to log in. On success you will be taken to the main dashboard.

Step 4 — Upload a file. Select Upload Media from the sidebar. Choose an image (JPG, PNG) or video (MP4, MOV) and click Upload. The system requests a presigned PUT URL, uploads the file directly to S3, then automatically runs deduplication, species detection, thumbnail generation, and stores the results in DynamoDB.

Step 5 — Query the library. Select Search Library. Choose a query mode — by species name, by tag counts (e.g. cat:2, bird:1), or by thumbnail URL. Results display thumbnail previews for

images and video thumbnails with a play indicator for videos. All media is served through time-limited presigned URLs — no direct S3 access is required.

Step 6 — Reverse search. Select Reverse Search and upload a query image. The system detects species in the image and returns all matching database files without storing the query file.

Step 7 — Manage tags. Select Manage Tags, paste one or more S3 file URLs, enter species tags, and choose Add or Remove. Bulk operations are supported.

Step 8 — Delete files. Select Delete Files, paste S3 file URLs, confirm the checkbox, and click Delete. The file, its thumbnail, and its DynamoDB record are all removed.

Step 9 — Notifications. Select Notifications, enter your email and the species names you want to watch, and click Subscribe. You will receive an SNS confirmation email. After confirming, you will receive alerts whenever a new file containing your watched species is uploaded.

4. Source Code

Repository: <https://github.com/bgan0012/AussieEcoLens>

The repository is private. All team members have committed code directly to the repository. Access has been granted to the full teaching team.

5. AI Tool Usage Declaration

This declaration outlines the use of AI tools in the completion of FIT5225 Assignment 2 (AussieEcoLens).

- Claude (Anthropic) was used to assist in writing and debugging Python Lambda functions across the AWS backend pipeline.
- ChatGPT was used to assist with frontend React component development and API service layer code.
- AI tools were not used to make architectural decisions — all design choices, service selections, and implementation strategies were determined by the team.
- All AI-generated code and content was reviewed, tested, and validated by team members before inclusion in the final submission.